Module Details

Module Code	**UFCFK1-15-0**
Module Title	**Fundamentals of Data Science**
Module Tutors	**Saurav Gautam**
Year	**2025-2026**
Component/Element Number	**PSA/Bi-weekly assignment/Regular**
Weighting	**10%**

Dates

Submission Date	**17-April-2026**
Submission Place	**Backboard**
Submission Time	**23:59**
Submission Notes	**Submit Gitlab URL**

Submitted By: Adarsha Karki

Student ID: 25068584

1. Write a program to read a file **“records.csv”** and display its contents. The fields to read are: student_name, roll_no, program, year, and group. **(10 marks)**

   => This program reads data from a file named records.csv and displays its contents on the screen. The file contains student details such as student name, roll number, program, year, and group. The program opens the CSV file in read mode, processes each line, and prints the data in a structured format. It helps in understanding how file handling works in Python, especially reading and displaying structured data from a CSV file.

```python
"""
Program to read records.csv and display contents (skipping header
row).
"""
try:
   with open("records.csv", "r") as file:
       next(file)  # skip header row
       for line in file:
           data = line.strip().split(",")
           print("Student Name:", data[0])
           print("Roll No:", data[1])
           print("Program:", data[2])
           print("Year:", data[3])
           print("Group:", data[4])
           print("-" * 30)
except FileNotFoundError:
   print("Error: records.csv file not found.")
except Exception as e:
   print("Unexpected error:", e)
```

   Output:

```
Student Name: Ads
Roll No: 101
Program: BSc CS
Year: 1
Group: A
-----------------------------
Student Name: Hari
Roll No: 102
Program: CsIT IT
Year: 2
Group: B
-----------------------------
Student Name: Ram
Roll No: 103
Program: BBA
Year: 1
Group: A
-----------------------------
```

:

2. Write a program to take user details as input and append them to the existing file **"records.csv"**. The fields to take input are: student_name, roll_no, program, year, and group. **(10 marks)**

   => This program takes student details (name, roll number, program, year, and group) from the user and validates that no field is empty. It then appends the data to records.csv using append mode without deleting existing records. The program also includes exception handling to manage file-related errors.

```python
"""
Append student details to records.csv with validation and error
handling.
"""
try:
   with open("records.csv", "a") as file:

       student_name = input("Enter student name: ").strip()
       while not student_name:
           print("Field cannot be empty")
           student_name = input("Enter student name: ").strip()

       roll_no = input("Enter roll number: ").strip()
       while not roll_no:
           print("Field cannot be empty")
           roll_no = input("Enter roll number: ").strip()

       program = input("Enter program: ").strip()
       while not program:
```

```
            print("Field cannot be empty")
            program = input("Enter program: ").strip()

        year = input("Enter year: ").strip()
        while not year:
            print("Field cannot be empty")
            year = input("Enter year: ").strip()

        group = input("Enter group: ").strip()
        while not group:
            print("Field cannot be empty")
            group = input("Enter group: ").strip()

file.write(f"{student_name},{roll_no},{program},{year},{group}\n")
        print("Record added successfully!")

except FileNotFoundError:
    print("Error: File not found.")

except PermissionError:
    print("Error: Permission denied.")

except Exception as e:
    print("Unexpected error:", e)
```

Output:

```
Enter student name:
Field cannot be empty
Enter student name: ads
Enter roll number: 1
Enter program: test
Enter year: 199
Enter group: d
Record added successfully!
```

3. Write a program to accept a list of integers from the user, perform **addition, subtraction, multiplication, and division** operations, and save the results into a file with the current date and time. The program should continue until the user decides to exit. When the user chooses to exit, the program should display the contents of the file in a neatly formatted way**. (10 marks)**

=>This program takes multiple numbers from the user and performs arithmetic operations such as addition, subtraction, multiplication, and division. . It stores the results along with the current date and time in a file named arithmetic_results.txt. Finally, it reads and displays the file contents. The program also uses exception handling to manage invalid inputs and file-related errors.

```
"""
Program to take numbers from user, perform arithmetic operations,
save results with datetime, and display file content.
"""

from datetime import datetime

file_name = "arithmetic_results.txt"

numbers = []
total_sum = 0
difference = None
product = 1
quotient = None

print("Enter numbers (type any non-number to stop):")

while True:
   try:
       num = input("Enter number: ")

       num = float(num)  # may raise ValueError
       numbers.append(num)

       # Addition
       total_sum += num

       # Subtraction
       if difference is None:
           difference = num
       else:
           difference -= num

       # Multiplication
```

```
        product *= num

        # Division
        if quotient is None:
            quotient = num
        else:
            if num != 0:
                quotient /= num
            else:
                print("Warning: Division by zero ignored.")

    except ValueError:
        break

# current datetime
current_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

# write results to file
try:
    with open(file_name, "a") as file:
        file.write("\n" + "=" * 50 + "\n")
        file.write(f"Date & Time: {current_time}\n")
        file.write(f"Numbers: {numbers}\n")
        file.write(f"Sum: {total_sum}\n")
        file.write(f"Difference: {difference}\n")
        file.write(f"Product: {product}\n")
        file.write(f"Quotient: {quotient}\n")
        file.write("=" * 50 + "\n")

    print("\nResults saved successfully!")

except Exception as e:
    print("File error:", e)

# display file content
try:
    print("\n--- File Content ---\n")
    with open(file_name, "r") as file:
        print(file.read())
```

```
except FileNotFoundError:
    print("File not found.")
```

Output:

```
erza@erza-HP-Pavilion-Laptop-14-dv2xxx:~/python/assignment3$ /usr/bin/python3 /home/
Enter numbers (type any non-number to stop):
Enter number: 1
Enter number: 2
Enter number: 4
Enter number: 56
Enter number: 5432
Enter number: -904
Enter number: q

Results saved successfully!

--- File Content ---


==================================================
Date & Time: 2026-04-13 17:32:28
Numbers: [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 0.0, 10.0]
Sum: 55.0
Difference: -53.0
Product: 0.0
Quotient: 2.7557319223985894e-07
==================================================

==================================================
Date & Time: 2026-04-16 19:39:47
Numbers: [1.0, 2.0, 4.0, 56.0, 5432.0, -904.0]
Sum: 4591.0
Difference: -4589.0
Product: -2199916544.0
Quotient: -4.5456269817479037e-10
==================================================
```

4. Write a program that reads from one file and copies the content into another file. The input and output file names should be entered by the user. Use **try/except** to handle exceptions. Display an error message if the input file does not exist, and also if the output file already exists. **(10 marks)**

=> This program reads content from a source file and copies it into a destination file. The user provides both file names as input. The program uses the os module to check if the output file already exists before copying. It also uses exception handling to manage errors such as missing input files. The source file is opened in read mode and the destination file is created in write mode to store the copied content.

```
"""Program to copy content from one file to another using exception
handling."""

import os

# Get input and output file names from the user

input_file = input("Enter the input file name: ").strip()

output_file = input("Enter the output file name: ").strip()

try:

    # Check if output file already exists before doing anything

    if os.path.exists(output_file):

        raise FileExistsError(f"Output file '{output_file}' already
exists.")

    # Try to open and read the input file

    with open(input_file, "r") as infile:

        content = infile.read()

    # Write content to the output file
```

```python
    with open(output_file, "w") as outfile:

        outfile.write(content)

    print(f"File copied successfully from '{input_file}' to
'{output_file}'.")

except FileNotFoundError:

    print(f"Error: Input file '{input_file}' does not exist.")

except FileExistsError as e:

    print(f"Error: {e}")

except Exception as e:

    print(f"An unexpected error occurred: {e}")
```

Output:

```
Enter the input file name: records.csv
Enter the output file name: file_copy.csv
File copied successfully from 'records.csv' to 'file_copy.csv'.
```

records.csv (source file):

```
cords.csv > data
You, 3 days ago | 1 author (You)
student_name,roll_no,program,year,group
Ads,101,BSc CS,1,A
Hari,102,CsIT IT,2,B        You, 3 days ago • co
Ram,103,BBA,1,Aads,1,test,199,d
```

file_copy.csv (destination file):

```
file_copy.csv > data
1   student_name,roll_no,program,year,group
2   Ads,101,BSc CS,1,A
3   Hari,102,CsIT IT,2,B
4   Ram,103,BBA,1,Aads,1,test,199,d
5
```

5. Create a class **Learner** with the attributes: roll_no, full_name, address, enrollment_year, program, and group. Instantiate an object of the class to take input for all attributes and display the details. **(10 marks)**

=>This program defines a class named Learner with attributes such as roll number, full name, address, enrollment year, program, and group. It takes user input to initialize these attributes and displays the stored student details. The program demonstrates basic object-oriented programming concepts such as class creation and object instantiation.

```
"""Program to create a Learner class, take user input, and display
details."""

class Learner:

   def __init__(self, full_name, roll_no, program, enrollment_year,
group, address):

       self.roll_no = roll_no

       self.full_name = full_name

       self.address = address

       self.enrollment_year = enrollment_year

       self.program = program

       self.group = group

   def __str__(self):
```

```
        return (f"\nName:             {self.full_name}"
                f"\nRoll No:         {self.roll_no}"
                f"\nProgram:         {self.program}"
                f"\nEnrollment Year: {self.enrollment_year}"
                f"\nGroup:           {self.group}"
                f"\nAddress:         {self.address}")

# Input validation functions
def validate_roll_no(prompt):
    """Validates that roll number is not empty and is numeric."""
    while True:
        value = input(prompt)
        if not value.strip():
            print("Roll number cannot be empty.")
            continue
        if not value.isdigit():
            print("Roll number must be numeric.")
            continue
        return int(value)
def validate_input(prompt):
    """Validates that input is not empty."""
    while True:
        value = input(prompt)
```

```
        if value.strip():
            return value
        else:
            print("Input cannot be empty. Please enter a valid value.")

# Function to get learner details from the user
def get_learner_info():
    """Takes all learner details from the user and returns a Learner object."""
    name = validate_input("Enter learner's full name: ")
    roll_no = validate_roll_no("Enter learner's roll number: ")
    program = validate_input("Enter learner's program: ")
    enrollment_year = validate_input("Enter learner's enrollment year: ")
    group = validate_input("Enter learner's group: ")
    address = validate_input("Enter learner's address: ")


    return Learner(name, roll_no, program, enrollment_year, group, address)

# Create a Learner object and display details
learner = get_learner_info()
print("\n--- Learner Details ---")
print(learner)
```

Output:

```
erza@erza-HP-Pavilion-Laptop-14-dv2xxx:~/python/assignment3$
Enter learner's full name: ads karki
Enter learner's roll number: 1
Enter learner's program: csit
Enter learner's enrollment year: 1999
Enter learner's group: a
Enter learner's address: Kapan

--- Learner Details ---

Name:             ads karki
Roll No:          1
Program:          csit
Enrollment Year:  1999
Group:            a
Address:          Kapan
erza@erza-HP-Pavilion-Laptop-14-dv2xxx:~/python/assignment3$
```

6. Write a program to create a class called **Dish** with attributes such as: dish_id, dish_name, ingredients, and instructions. Create another class called **Cookbook** to manage a collection of dishes. **(10 marks)**

=> This program creates a Dish class to represent individual dishes with attributes such as dish ID, dish name, ingredients, and instructions. It also creates a CookBook class to manage a collection of dishes. The program allows the user to add multiple dishes and stores them in a list. Finally, it displays all stored dishes. This program demonstrates object-oriented programming concepts such as classes, objects, and collection management.

```
"""
Program to create a Dish class and manage a collection of dishes
using Cookbook class.
"""

# Class representing a single Dish
class Dish:
   def __init__(self, dish_id, dish_name, ingredients,
instructions):
```

```python
        self.dish_id = dish_id
        self.dish_name = dish_name
        self.ingredients = ingredients
        self.instructions = instructions

    def __str__(self):
        return (f"Dish ID: {self.dish_id}\n"
                f"Dish Name: {self.dish_name}\n"
                f"Ingredients: {', '.join(self.ingredients)}\n"
                f"Instructions: {self.instructions}\n")


# Class representing the Cookbook
class CookBook:
    def __init__(self):
        self.dishes = []

    def add_dish(self, dish):
        self.dishes.append(dish)

    def display_dishes(self):
        if not self.dishes:
            print("No dishes in the cookbook.")
        else:
            for dish in self.dishes:
                print(dish)

# Input validation functions
def validate_dish_id(prompt):
    while True:
        value = input(prompt)
        if value.lower() == 'exit':
            return None
        if not value.strip():
            print("Dish ID cannot be empty.")
            continue
        if not value.isdigit():
            print("Dish ID must be numeric.")
            continue
        return int(value)
```

```python
def validate_input(prompt):
    while True:
        value = input(prompt)
        if value.strip():
            return value
        else:
            print("Input cannot be empty. Please enter a valid value.")

# Main program
cookbook = CookBook()

while True:
    dish_id = validate_dish_id("Enter dish ID (or 'exit' to stop): ")
    if dish_id is None:
        break

    dish_name = validate_input("Enter dish name: ")

    # Ingredients: remove empty entries and extra spaces
    ingredients = [i.strip() for i in validate_input(
        "Enter ingredients (comma-separated): ").split(',') if i.strip()]

    instructions = validate_input("Enter instructions: ")

    dish = Dish(dish_id, dish_name, ingredients, instructions)
    cookbook.add_dish(dish)

# Display all dishes in the cookbook
print("\nCookbook:")
cookbook.display_dishes()
```

Output:

```
erza@erza-HP-Pavilion-Laptop-14-dv2xxx:~/python/assignment3$ /usr/bin/python3 /home/erza/python/assignmer
Enter dish ID (or 'exit' to stop): 1
Enter dish name: fish
Enter ingredients (comma-separated): sushi, soy-souce , tomato, onion
Enter instructions: use sushi when gas is hot and then put the souce and other seasonings (tomato, onion)
Enter dish ID (or 'exit' to stop): 2
Enter dish name: applepie
Enter ingredients (comma-separated): apple, honey
Enter instructions: in oven keep apple and honey
Enter dish ID (or 'exit' to stop): exit

Cookbook:
Dish ID: 1
Dish Name: fish
Ingredients: sushi, soy-souce, tomato, onion
Instructions: use sushi when gas is hot and then put the souce and other seasonings (tomato, onion)

Dish ID: 2
Dish Name: applepie
Ingredients: apple, honey
Instructions: in oven keep apple and honey
```

7. Write a program to create a class called **Staff** with attributes such as: emp_id, full_name, address, phone_number, marital_status, dependents, and salary. Instantiate this class for multiple staff members and write the data into a file **"staff.csv"**. The program should also allow the user to view the list of staff and their details. Use the concept of **try/except** in the program. **(20 marks)**

   =>This program creates a Staff class to store employee details such as employee ID, full name, address, phone number, marital status, dependents, and salary. It allows the user to enter multiple staff records, which are stored in a list and saved into a CSV file named staff.csv. The program uses file handling to write and read data, and exception handling to manage errors such as file not found or input issues. It also displays all stored staff records in a structured format.

```
"""
Program to create Staff class, store multiple staff records, save to
CSV file,
and display them with file handling and exception handling.
```

```python
"""

import os
file_name = "staff.csv"

class Staff:
    def __init__(self, emp_id, full_name, address, phone_number,
marital_status, dependents, salary):
        self.emp_id = emp_id
        self.full_name = full_name
        self.address = address
        self.phone_number = phone_number
        self.marital_status = marital_status
        self.dependents = dependents
        self.salary = salary

    def __str__(self):
        return (f"  Employee ID    : {self.emp_id}\n"
                f"  Full Name      : {self.full_name}\n"
                f"  Address        : {self.address}\n"
                f"  Phone Number   : {self.phone_number}\n"
                f"  Marital Status : {self.marital_status}\n"
                f"  Dependents     : {self.dependents}\n"
                f"  Salary         : {self.salary}")

    def to_csv_line(self):
        """Returns a CSV-formatted line for this staff member."""
        return
f"{self.emp_id},{self.full_name},{self.address},{self.phone_number},
{self.marital_status},{self.dependents},{self.salary}\n"


# Validation helpers

def validate_input(prompt):
    """Validates that a text field is not empty."""
    while True:
        value = input(prompt).strip()
        if value:
            return value
```

```
        print("  Error: This field cannot be empty.")

def validate_salary(prompt):
    """Validates that salary is a valid positive number."""
    while True:
        value = input(prompt).strip()
        if not value:
            print("  Error: Salary cannot be empty.")
            continue
        try:
            salary = float(value)
            if salary < 0:
                print("  Error: Salary cannot be negative.")
                continue
            return salary
        except ValueError:
            print("  Error: Salary must be a valid number.")

def validate_dependents(prompt):
    """Validates that dependents is a non-negative whole number."""
    while True:
        value = input(prompt).strip()
        if not value:
            print("  Error: Dependents cannot be empty.")
            continue
        if not value.isdigit():
            print("  Error: Dependents must be a whole number (0 or
more).")
            continue
        return int(value)

# Core functions
def save_to_file(staff_list):
    """Writes all staff records to staff.csv with proper header
handling."""
    try:
        file_exists = os.path.exists(file_name)

        with open(file_name, "a") as f:
            # write header only once
```

```
            if not file_exists:

f.write("emp_id,full_name,address,phone_number,marital_status,depend
ents,salary\n")

            for staff in staff_list:
                f.write(staff.to_csv_line())

    except IOError as e:
        print(f"Error writing file: {e}")

def display_all_records():
    """Reads and displays all records from staff.csv. Uses try/except
for file errors."""
    try:
        with open(file_name, "r") as f:
            lines = f.readlines()[1:]  # skip header row

        print("\n" + "=" * 50)
        print("           ALL STAFF RECORDS")
        print("=" * 50)

        for i, line in enumerate(lines, start=1):
            data = line.strip().split(",")
            print(f"\n  Record #{i}")
            print(f"  Employee ID    : {data[0]}")
            print(f"  Full Name      : {data[1]}")
            print(f"  Address        : {data[2]}")
            print(f"  Phone Number   : {data[3]}")
            print(f"  Marital Status : {data[4]}")
            print(f"  Dependents     : {data[5]}")
            print(f"  Salary         : {data[6]}")
            print("  " + "-" * 40)

    except FileNotFoundError:
        print(f"  Error: '{file_name}' not found.")
    except IOError as e:
        print(f"  Error: Could not read file. {e}")

# Main program
```

```
staff_list = []

print("Enter staff details. Type 'done' as Employee ID to stop.\n")

while True:
   print("--- Enter Staff Details ---")
   emp_id_input = input("Enter employee ID (or 'done' to stop): ").strip()

   # User intentionally wants to stop
   if emp_id_input.lower() == "done":
       break

   if not emp_id_input.isdigit() or int(emp_id_input) <= 0:
       print("  Error: Employee ID must be a positive number. Try again.")
       continue

   try:
       full_name  = validate_input("Enter full name       : ")
       address    = validate_input("Enter address         : ")
       phone      = validate_input("Enter phone number    : ")
       marital    = validate_input("Enter marital status  : ")
       dependents = validate_dependents("Enter dependents      : ")
       salary     = validate_salary("Enter salary          : ")

       staff = Staff(int(emp_id_input), full_name, address, phone, marital, dependents, salary)
       staff_list.append(staff)
       save_to_file(staff_list)
       print("\n  Staff member added successfully.\n")

   except Exception as e:
       print(f"  Unexpected error: {e}")

# Display all records after the user is done entering
if staff_list:
   display_all_records()
else:
```

```
    print("\n  No staff members were added.")
```

Output:

```
erza@erza-HP-Pavilion-Laptop-14-dv2xxx:~/python/assignment3$ /usr/bin/python3 /home/erza/python/assignmen
Enter staff details. Type 'done' as Employee ID to stop.

--- Enter Staff Details ---
Enter employee ID (or 'done' to stop): 1
Enter full name       : ads karki
Enter address         : kapan
Enter phone number    : 9834567890
Enter marital status  : yes
Enter dependents      : 4
Enter salary          : 99999999

  Staff member added successfully.

--- Enter Staff Details ---
Enter employee ID (or 'done' to stop): done

==================================================
           ALL STAFF RECORDS
==================================================

  Record #1
  Employee ID    : 1
  Full Name      : ads karki
  Address        : kapan
  Phone Number   : 9834567890
  Marital Status : yes
  Dependents     : 4
  Salary         : 99999999.0
```

8. Write a program to design a basic **library management system** with functionalities such as: **borrow a book, return a book, and search for a book**. Use OOP concepts to create necessary classes and apply OOP features (like encapsulation/inheritance if applicable). For storing book details, use file handling along with exception handling. **(20 marks)**

=>This program implements a library management system using object-oriented programming. It includes classes such as Book, EBook, User, and Library to manage books and users. The system allows users to search for books, borrow books, return books, and register new users. It also uses file handling to load and save book and user data. Exception handling is used to manage errors such as unavailable books and missing files.

```
# Custom Exception

class BookNotAvailableError(Exception):
```

```
    """Raised when a user tries to borrow a book that is not
available."""

    pass

# Book class

class Book:

    """Represents a physical book in the library."""

    def __init__(self, book_id, title, author, is_available=True):

        self.__book_id      = book_id        # private

        self.__title        = title          # private

        self.__author       = author         # private

        self.__is_available = is_available   # private

    # read-only properties

    @property

    def book_id(self):

        return self.__book_id

    @property

    def title(self):

        return self.__title

    @property

    def author(self):

        return self.__author

    @property
```

```
    def is_available(self):
        return self.__is_available
    # setter with validation
    @is_available.setter
    def is_available(self, value):
        if not isinstance(value, bool):
            raise ValueError("is_available must be True or False.")
        self.__is_available = value
    def get_details(self):
        status = "Available" if self.__is_available else "Not
Available"
        return (f"ID: {self.__book_id} | Title: {self.__title} "
                f"| Author: {self.__author} | Status: {status}")
    def borrow_book(self):
        if not self.__is_available:
            raise BookNotAvailableError(f"'{self.__title}' is
currently not available.")
        self.__is_available = False
    def return_book(self):
        self.__is_available = True
# EBook class (Inheritance)
class EBook(Book):
    """Inherits from Book.
```

```
    Adds digital-specific attributes: file format and download URL.

    """

    def __init__(self, book_id, title, author, file_format,
download_url, is_available=True):

        super().__init__(book_id, title, author, is_available)  #
call parent constructor

        self.__file_format  = file_format

        self.__download_url = download_url

    @property

    def file_format(self):

        return self.__file_format



    @property

    def download_url(self):

        return self.__download_url

    def get_details(self):                    # method overriding

        base = super().get_details()

        return f"{base} | Format: {self.__file_format} | URL:
{self.__download_url}"

# User class

class User:

    """Represents a library member."""

    def __init__(self, user_id, name):
```

```
        self.user_id        = user_id

        self.name           = name

        self.borrowed_books = []

    def borrow_book(self, book):

        book.borrow_book()                  # raises
BookNotAvailableError if unavailable

        self.borrowed_books.append(book)

    def return_book(self, book):

        if book in self.borrowed_books:

            book.return_book()

            self.borrowed_books.remove(book)

            return True

        return False

# Library class

class Library:

    """Manages books and users. Handles file storage."""

    def __init__(self):

        self.books = []

        self.users = []

    # search

    def search_book(self, title):

        for book in self.books:
```

```
        if title.lower() in book.title.lower():
            return book
    return None
def search_user(self, name):
    for user in self.users:
        if name.lower() in user.name.lower():
            return user
    return None
# borrow / return
def borrow_book(self, user_name, book_title):
    user = self.search_user(user_name)
    book = self.search_book(book_title)
    if not user or not book:
        print("User or Book not found.")
        return
    try:
        user.borrow_book(book)
        print(f"'{user.name}' has borrowed '{book.title}'.")
    except BookNotAvailableError as e:
        print(f"Error: {e}")
def return_book(self, user_name, book_title):
    user = self.search_user(user_name)
```

```
        book = self.search_book(book_title)

        if not user or not book:

            print("User or Book not found.")

            return

        if user.return_book(book):

            print(f"'{user.name}' has returned '{book.title}'.")

        else:

            print(f"'{user.name}' did not borrow '{book.title}'.")

    # display

    def display_books(self):

        if not self.books:

            print("No books available in the library.")

            return

        print("\n--- All Books ---")

        for book in self.books:

            print(book.get_details())

    def display_users(self):

        if not self.users:

            print("No users registered.")

            return

        print("\n--- All Users ---")

        for user in self.users:
```

```
            borrowed = [b.title for b in user.borrowed_books]

            print(f"ID: {user.user_id} | Name: {user.name} |
Borrowed: {borrowed}")

    # register user

    def register_user(self, user_id, name):

        if self.search_user(name):

            print(f"User '{name}' already exists.")

        else:

            self.users.append(User(user_id, name))

            print(f"User '{name}' registered successfully.")

    # file handling

    def load_books_from_file(self, file_path):

        try:

            with open(file_path, "r") as f:

                for line in f:

                    parts = line.strip().split(",")

                    if len(parts) != 4:

                        continue                    # skip malformed
lines

                    book_id, title, author, is_available = parts

                    self.books.append(Book(book_id, title, author,
is_available == "True"))

            print(f"Books loaded from '{file_path}'.")
```

```
        except FileNotFoundError:
            print(f"'{file_path}' not found. Starting with empty book
list.")
        except ValueError as e:
            print(f"Error reading book file: {e}")
    def load_users_from_file(self, file_path):
        try:
            with open(file_path, "r") as f:
                for line in f:
                    parts = line.strip().split(",")
                    if len(parts) != 2:
                        continue
                    user_id, name = parts
                    self.users.append(User(user_id, name))
            print(f"Users loaded from '{file_path}'.")
        except FileNotFoundError:
            print(f"'{file_path}' not found. Starting with empty user
list.")
        except ValueError as e:
            print(f"Error reading user file: {e}")
    def save_books_to_file(self, file_path):
        try:
            with open(file_path, "w") as f:
```

```
                for book in self.books:

f.write(f"{book.book_id},{book.title},{book.author},{book.is_availab
le}\n")

            print(f"Books saved to '{file_path}'.")

        except OSError as e:

            print(f"Could not save books: {e}")

    def save_users_to_file(self, file_path):

        try:

            with open(file_path, "w") as f:

                for user in self.users:

                    f.write(f"{user.user_id},{user.name}\n")

            print(f"Users saved to '{file_path}'.")

        except OSError as e:

            print(f"Could not save users: {e}")

# menu and main

def menu():

    print("\n========== Library Menu ==========")

    print("1. Display all books")

    print("2. Search a book")

    print("3. Borrow a book")

    print("4. Return a book")

    print("5. Register a new user")
```

```
    print("6. Display all users")

    print("7. Exit")

    print("==================================")

def main():

    library = Library()

    library.load_books_from_file("books.txt")

    library.load_users_from_file("users.txt")

    while True:

        menu()

        choice = input("Enter your choice: ").strip()



        if choice == "1":

            library.display_books()

        elif choice == "2":

            title = input("Enter book title to search: ").strip()

            book = library.search_book(title)

            if book:

                print(book.get_details())

            else:

                print("Book not found.")

        elif choice == "3":

            name  = input("Enter your name: ").strip()
```

```
            title = input("Enter book title: ").strip()

            library.borrow_book(name, title)

        elif choice == "4":

            name  = input("Enter your name: ").strip()

            title = input("Enter book title: ").strip()

            library.return_book(name, title)

        elif choice == "5":

            uid  = input("Enter user ID: ").strip()

            name = input("Enter user name: ").strip()

            library.register_user(uid, name)



        elif choice == "6":

            library.display_users()

        elif choice == "7":

            library.save_books_to_file("books.txt")

            library.save_users_to_file("users.txt")

            print("Goodbye!")

            break

        else:

            print("Invalid choice. Please try again.")

if __name__ == "__main__":

    main()
```

Output:

Register a user:

```
erza@erza-hp-Pavilion-Laptop-14-dv2xxx:~/python/assignment3$ /usr/bin/python3 /home/erza/python/assignment
'books.txt' not found. Starting with empty book list.
'users.txt' not found. Starting with empty user list.

========== Library Menu ==========
1. Display all books
2. Search a book
3. Borrow a book
4. Return a book
5. Register a new user
6. Display all users
7. Exit
==================================
Enter your choice: 5
Enter user ID: 1
Enter user name: ads
User 'ads' registered successfully.
```

Display all books:

```
========== Library Menu ==========
1. Display all books
2. Search a book
3. Borrow a book
4. Return a book
5. Register a new user
6. Display all users
7. Exit
==================================
Enter your choice: 1

--- All Books ---
ID: 1 | Title: Python Basics | Author: John Zelle | Status: Available
ID: 2 | Title: Data Structures | Author: Mark Weiss | Status: Available
ID: 3 | Title: Machine Learning | Author: Andrew Ng | Status: Not Available
ID: 4 | Title: Operating Systems | Author: Galvin | Status: Available
ID: 5 | Title: Database Systems | Author: Elmasri | Status: Available
ID: 6 | Title: Computer Networks | Author: Andrew Tanenbaum | Status: Available
ID: 7 | Title: Artificial Intelligence | Author: Stuart Russell | Status: Not Available
```

Search a book:

```
========== Library Menu ==========
1. Display all books
2. Search a book
3. Borrow a book
4. Return a book
5. Register a new user
6. Display all users
7. Exit
==================================
Enter your choice: 2
Enter book title to search: py
ID: 1 | Title: Python Basics | Author: John Zelle | Status: Available
```

Borrow a book:

(when available)

```
========== Library Menu ==========
1. Display all books
2. Search a book
3. Borrow a book
4. Return a book
5. Register a new user
6. Display all users
7. Exit
==================================
Enter your choice: 3
Enter your name: ads
Enter book title: py
'ads' has borrowed 'Python Basics'.
```

(when not available)

```
'ads' has borrowed 'Python Basics'.

========== Library Menu ==========
1. Display all books
2. Search a book
3. Borrow a book
4. Return a book
5. Register a new user
6. Display all users
7. Exit
==================================
Enter your choice: 3
Enter your name: ads
Enter book title: python basics
Error: 'Python Basics' is currently not available.
```

Return a book:

```
========== Library Menu ==========
1. Display all books
2. Search a book
3. Borrow a book
4. Return a book
5. Register a new user
6. Display all users
7. Exit
==================================
Enter your choice: 4
Enter your name: ads
Enter book title: python basics
'ads' has returned 'Python Basics'.
```

Display all users:

```
========== Library Menu ==========
1. Display all books
2. Search a book
3. Borrow a book
4. Return a book
5. Register a new user
6. Display all users
7. Exit
==================================
Enter your choice: 6

--- All Users ---
ID: 1 | Name: ads | Borrowed: ['Data Structures', 'Operating Systems']
```

On invalid inputs:

```
========== Library Menu ==========
1. Display all books
2. Search a book
3. Borrow a book
4. Return a book
5. Register a new user
6. Display all users
7. Exit
==================================
Enter your choice: 9
Invalid choice. Please try again.
```

```
========== Library Menu ==========
1. Display all books
2. Search a book
3. Borrow a book
4. Return a book
5. Register a new user
6. Display all users
7. Exit
==================================
Enter your choice: 3
Enter your name: data
Enter book title: data
User or Book not found.
```

Exit:

```
========== Library Menu ==========
1. Display all books
2. Search a book
3. Borrow a book
4. Return a book
5. Register a new user
6. Display all users
7. Exit
==================================
Enter your choice: 7
Books saved to 'books.txt'.
Users saved to 'users.txt'.
Goodbye!
```

Sample data files:

books.txt:

books.txt

```
1,Python Basics,John Zelle,True
2,Data Structures,Mark Weiss,False
3,Machine Learning,Andrew Ng,False
4,Operating Systems,Galvin,False
5,Database Systems,Elmasri,True
6,Computer Networks,Andrew Tanenbaum,True
7,Artificial Intelligence,Stuart Russell,False
```

users.txt

users.txt

```
1,ads
```

records.csv

records.csv > data

You, 43 minutes ago | 1 author (You)

```
student_name,roll_no,program,year,group
Ads,101,BSc CS,1,A
Hari,102,CsIT IT,2,B
Ram,103,BBA,1,Aads,1,test,199,d
```

staff.csv:

staff.csv > data

You, 14 minutes ago | 1 author (You)

```
emp_id,full_name,address,phone_number,marital_status,dependents,salary
1,ads karki,kapan,9834567890,yes,4,99999999.0
```

file_copy.csv:

file_copy.csv > data

```csv
student_name,roll_no,program,year,group
Ads,101,BSc CS,1,A
Hari,102,CsIT IT,2,B
Ram,103,BBA,1,Aads,1,test,199,d
```